qzone-skill 开发者手册

v1.0 · 架构、实现取舍、扩展点、已知限制

一、总览

```
CLI (scripts/qzone.mjs)
 │
 ├─ store.mjs ──────── 配置 / 状态 / 暂停标记          纯本地，无网络
 ├─ login.mjs ──────── 扫码登录、登录态落盘
 ├─ service.mjs ────── 系统定时任务（wscript + schtasks）
 └─ actions.mjs ────── 读动态 / 点赞 / 自动刷
      │
      ├─ browser.mjs ─ 启动·复用·关闭 headless Edge（独立 profile）
      └─ cdp.mjs ────── 极简 CDP 客户端（fetch + 内置 WebSocket）
                          │
                          ▼
                  Edge (headless)
                          │  页面内 fetch / DOM 操作
                          ▼
                  user.qzone.qq.com
```

零运行时依赖。Node 22+ 提供内置 `WebSocket` 和 `fetch`，够用了。

文件	职责	行数
`scripts/qzone.mjs`	CLI 入口、参数解析、命令分发	~200
`lib/cdp.mjs`	CDP 连接、RPC、事件、截图、导航	~200
`lib/browser.mjs`	Edge 的启动/复用/关闭，UA 覆盖	~140
`lib/login.mjs`	扫码、轮询、登录态持久化	~150
`lib/actions.mjs`	DOM 抓取、点赞、规则、自动刷循环	~300
`lib/service.mjs`	计划任务安装与查询	~110
`lib/store.mjs`	配置合并、点号读写、类型转换、状态	~180

二、核心取舍：为什么走浏览器，不走接口

QQ 空间有一批「看起来能用」的内部接口，但全部带签名参数：

参数	来源	失效特征
`g_tk`	由 `p_skey` cookie 做 djb2 哈希算出	算法本身稳定，但 cookie 一换就得重算

`curkey`	动态列表返回体里，每条一个	列表接口一改，就取不到
`unikey`	点赞接口要的，形如 `http://user.qzone.qq.com/<uin>/mood/<tid>`	tid 编码规则变过
`qzonetoken`	页面初始化注入	页面改版就变

这些参数里任何一个算错，接口的返回仍然是 `200`，但 `code` 字段是业务错误码 —— 也就是说**失败是静默的。**逆向实现的典型寿命是几个月。

选浏览器路线的三个理由

1. **签名由浏览器自己算。**页面里发起的请求，cookie、referer、签名、甚至某些 JS 挑战都是对的，因为它就是页面自己。
2. **失败是可见的。**点赞就是在点那个按钮，点完看 class 有没有变成 `_on`。改版了立刻能发现，而不是「接口返回成功但实际没赞上」。
3. **改版只需要换选择器。**DOM 结构变化比签名算法变化慢得多，而且有 `explore` 命令能直接看到当前结构。

> **不选 puppeteer / playwright 的理由**：要下载 300MB+ 的 Chromium；本项目只需要 CDP 的 Browser / Page / Runtime 三组域，用内置 WebSocket 写 60 行就够。用户拿到仓库直接能跑，不会卡在 `npm install`。

三、CDP 客户端（cdp.mjs）

3.1 连接流程

```
spawn(msedge, ["--remote-debugging-port=0", "--user-data-dir=...", ...])
   │
   ├─ 浏览器启动完成后写 <profile>/DevToolsActivePort（第一行 = 端口）
   │
   ├─ GET http://127.0.0.1:<port>/json/version  →  webSocketDebuggerUrl
   │
   └─ new WebSocket(wsUrl)  →  Cdp 实例
```

用 `--remote-debugging-port=0` 让系统分配端口，而不是写死 9222。写死端口会在「用户已开着另一个调试实例」时撞车，而且并发跑两份会互相踩。

3.2 新开页面

```
PUT /json/new?<url>              →  { id, webSocketDebuggerUrl }
Target.attachToTarget            →  { sessionId }       // flatten: true
Page.enable / Runtime.enable / Network.enable
```

注意：`/json/new` 必须用 `PUT`，用 `GET` 会返回 `405`。这是 Chrome 80 之后改的。

3.3 执行页面脚本

```
Runtime.evaluate {
  expression, awaitPromise: true, returnByValue: true, userGesture: true
}
```

- `userGesture: true` 很关键。有些点击在无用户手势上下文里会被浏览器忽略，点赞按钮就属于这类。
- `returnByValue` 让结果直接是 JSON，不用再 `Runtime.getProperties` 递归取。
- `exceptionDetails` 非空时必须抛错 —— 否则页面脚本报错会静默返回 `undefined`，看起来像「读不到动态」。

3.4 事件与超时

事件按 `sessionId:method` 分组。每个 RPC 有 60 秒超时兜底：浏览器的某些调用（比如 `Browser.close`）可能永远不回。

四、浏览器生命周期（browser.mjs）

4.1 为什么用独立 profile

`--user-data-dir` 指向 `%APPDATA%\qzone-skill\browser`，而不是用户平时的 Edge 配置：

- 不碰用户的标签页、书签、扩展

- 不会因为用户开着 Edge 而抢 `SingletonLock`（Chrome 系浏览器的 profile 是单实例独占的）
- 「换号」= 删这个文件夹；「备份登录态」= 拷这个文件夹

> **踩过的坑**：一开始想复用用户 Edge 里已有的登录态，直接读 `User Data\Default\Network\Cookies`。实测在 Edge 运行期间用 Node 读会得到 `EBUSY`（Chromium 用独占句柄打开这个文件），`Copy-Item` 同样失败。加上新版 Chromium 的 App-Bound Encryption 会把 cookie 密钥绑定到应用路径，就算复制出来也大概率解不开。⇒ **结论：不要试图偷 cookie，老老实实扫码。**

4.2 实例复用

每次 `ensureBrowser()` 先读 `DevToolsActivePort`，能连上就复用，省 2~3 秒启动时间，也避免攒出一堆僵尸实例。连不上才 spawn 新的。

4.3 UA 覆盖（必做）

```
--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/156.0.0.0 Safari/537.36 Edg/156.0.0.0
```

headless 模式默认 UA 里带 `HeadlessChrome`，很多站点会直接返回拦截页。必须显式覆盖成普通 Edge 的写法。

4.4 关闭

发 `Browser.close` 而不是 kill 进程。kill 会让 profile 留下脏锁文件，下次启动可能报「profile 正在使用」。

五、登录（login.mjs）

5.1 流程

1. 打开 `https://user.qzone.qq.com/`
2. 已登录 → URL 会被改写成 `/<uin>`，正则 `user\.qzone\.qq\.com\/(\d{5,})` 命中即视为成功
3. 未登录 → 页面里嵌一个 `xui.ptlogin2.qq.com` 的 iframe，二维码在里面
4. 按 iframe 的 `getBoundingClientRect()` 裁剪截图（不写死坐标，抗改版）
5. 每 2 秒查一次 URL；每 25 秒重截一次二维码
6. 成功后把 `uin` 和昵称写进 `config.json`

5.2 为什么不用 ptlogin2 的扫码协议

理论上可以纯 HTTP 拿到二维码：请求 `ptqrshow` 拿 PNG，用 `hash33(qrsig)` 算 `ptqrtoken`，再轮询 `ptqrlogin`。仓库里早期确实写过这套（`make-qr.mjs` / `poll-login.mjs`）。

放弃的原因是：**这套协议不带浏览器指纹。**扫码成功后拿到的 cookie 是「无头登录」的形态，访问空间时经常被要求二次验证或直接判为异常。而走真实浏览器渲染的登录页，拿到的是「正常网页登录」的会话，稳定得多。

5.3 登录态的存放 —— 这里有一个非直觉的坑

第一版我以为「登录态存在浏览器 profile 里，下次起浏览器自动带上」。实测**这是错的**。

登录成功后查 cookie 库：

```
.qzone.qq.com  p_skey   persistent=0   ← 0 = session cookie
.qzone.qq.com  p_uin    persistent=0
.qq.com        skey     persistent=0
.qq.com        uin      persistent=0
```

全是 session cookie。Chromium 只把它们放在内存里，进程一退就删——profile 目录根本存不住。表现就是：扫码明明成功了，下一条命令却跳回登录页。

解法：导出 + 注入

```
login 成功 ──▶ Storage.getCookies ──▶ session.json（磁盘）
                                            │
每次起浏览器 ──▶ Storage.setCookies ◀──────┘（expires 从 -1 改成 30 天后）
```

代码在 `lib/session.mjs`：`exportSession()` 在登录成功的那一刻抓，`injectSession()` 在每次导航**之前**注入。

> **注入时有两个必须处理的细节**：
> ① `SameSite=None` 必须同时 `Secure`，否则 Chromium 整批拒绝，报 `Invalid cookie fields`；
> ② 批量注入时只要一条不合法，整批失败且看不出是哪条。所以改成逐条注入并收集失败项。

另一条弯路（记下来免得再走）

为了省一次扫码，我试过直接从 `Cookies` sqlite 里解密取 `p_skey`：`v10` 前缀 + AES-256-GCM，密钥由 DPAPI 保护、从 `Local State` 的 `os_crypt.encrypted_key` 解出。都成功了。

但解出来的「值」是 32 字节乱码——因为 **新版 Chromium 在解密结果前面垫了 32 字节 `sha256(host_key)`**。切掉这 32 字节才是真正的值。我当时没切，把它当成 `p_skey` 存进了 session.json，注入时报错才发现。

而且这条路本身也没用：**浏览器一退出，Chromium 就把 session cookie 的记录从库里删了**，事后再解密只能拿到残留。所以别绕，老老实实「登录时抓、用时注」。

5.4 别指望复用用户自己 Edge 的登录态

- Edge 运行时 `User Data\Default\Network\Cookies` 被独占打开，Node 读报 `EBUSY`，`Copy-Item` 也失败；
- 新版 Chromium 的 App-Bound Encryption 把密钥绑到应用路径，就算复制出来也大概率解不开；
- 综合下来：**扫码是唯一稳的路**，而且只扫一次。

> **安全边界**：`session.json` 与 `browser` 目录等同账号。代码里没有任何地方会把它们打印出来或发到网络上。加功能时也请守住这条。

六、读动态与点赞（actions.mjs）

6.1 抓取脚本与真实 DOM 结构

在页面里执行（`Runtime.evaluate`），返回 `{ items, probe }`。2026-11 实测的结构：

```
li.f-single  id="fct_1987417704_311_5_..."
├─ .f-single-head
│   ├─ .f-nick .f-name            ← 发布者昵称
│   └─ .info-detail .state        ← 时间（"12:45"）
└─ .f-single-content
    └─ .f-item[data-key="6896..."]   ← 真正的动态 id
        └─ .qz_summary i[name="feed_data"][data-tid]  ← tid 在这一层
            └─ .txt-box              ← 正文（转发的正文也在这）
                └─ a.nickname        ← 原作者名，正文前面会带上它
    └─ .f-single-foot            ← 赞按钮在这一层
```

几个容易踩错的地方：

陷阱	说明
li 的 `id` 不是动态 id	它是 `fct_<uid>_...` 形式的卡片 id。点赞要用的 `data-key` 在内层 `.f-item` 上。
`data-tid` 不在 `.f-item` 上	它挂在更内层的 `<i name="feed_data">` 上，得单独查。
正文不在 `.f-info` 里	纯文字和转发都在 `.txt-box`。只查 `.f-info` 会得到一堆空字符串。
时间不在 `.f-time` 里	在 `.info-detail .state`，格式是 `HH:mm`（当天的），更早的会带日期。

转发的正文开头带原作者名	`.txt-box` 的 `innerText` 形如「飞你爹：正文」，要把开头的 `a.nickname` 剪掉。
广告卡片没有 `data-key`	「热血世界」「散人打金服」这类是空间塞的推广位，没有动态 id，点赞无从谈起，直接跳过。

`probe.candidates` 会返回每个选择器的命中数，改版时先看它：`li.f-single` 有值但 `.f-item[data-key]` 是 0，说明 id 的位置变了；两个都是 0，说明没登录或页面没加载完。

6.2 时间换算

`ageHoursFromText()` 把「3分钟前 / 昨天 12:34 / 10月30日 / 2026-10-30」统一成小时数。**认不出时返回 `null`，而不是 0** —— 返回 0 会被当成「刚刚发的」，导致补赞一堆老动态。调用方对 `null` 的处理是「不做年龄判断，交给页面上的已赞状态兜底」。

6.3 规则判定

`shouldLike()` 的顺序是有意的：

1. 本地去重（`state.liked`）
2. 页面已赞
3. `denyUsers`（黑名单优先于白名单）
4. `allowUsers`（非空时才生效）
5. `denyKeywords`
6. 年龄

黑名单优先于白名单，是因为「我明确讨厌的人」比「我关注的人」更该被尊重。

6.4 点赞的三种失败模式

返回	原因	正确处理
`ok:true, skipped:true`	class 里已有 `_on`	跳过，不重复点
`ok:false, "找不到赞按钮"`	卡片结构变了，或者这条动态类型特殊	跳过这一条，继续别的
`ok:false, "点了但状态没变"`	风控 / 需要先展开 / 按钮位置错	**停止这一轮**，不要重试

点击后固定等 1500ms 再读 class。实测这个延时足够覆盖网络往返；调小会出现「本来成功了却读成失败」。

按钮怎么定位：`data-key` 在 `.f-item` 上，而赞按钮在更外层的 `.f-single-foot` 里。所以不能直接在被点元素上 `querySelector`，要先 `closest('li.f-single')` 往上走一层再找按钮：

```
const keyEl = [...document.querySelectorAll('[data-key]')].find(n => n.getAttribute('data-key') === KEY);
const card  = keyEl.closest('li.f-single') || keyEl;
const btn   = card.querySelector('[class*="like_btn"]');
```

6.5 限速

两条之间 `auto.like.minIntervalSec`（默认 8 秒），代码里再夹一个 1000ms 下限兜底。这个值不是随便定的：短时间内连续点赞是 QQ 空间风控最敏感的行为。

七、配置与状态（store.mjs）

7.1 配置合并

`loadConfig()` 用「默认值 ← 用户文件」的深合并，数组整体替换而不是拼接。好处是新版本加了配置项，老用户的配置文件缺字段也不会崩。

7.2 点号读写与类型

`setPath(obj, 'auto.like.maxPerRun', '5')` 会按**原值的类型**转换：

原值	输入	结果
`10` (number)	`"5"`	`5`
`false` (boolean)	`"1"` / `"0"`	`true` / `false`
`[]` (array)	`"a,b"`	`["a","b"]`

为什么按原值而不是按输入猜：命令行只给字符串，按输入猜的话 `"123"` 到底是字符串还是数字没法判断。

7.3 状态文件

`state.json` 记 `liked`（key → {at, name, text}），最多 500 条，超了按时间淘汰最老的。文件坏了就当空的——去重丢一次比整个工具起不来强。

八、定时任务（service.mjs）

8.1 Windows 上的隐藏执行

```
schtasks  →  wscript.exe "run-auto.vbs"  →  node qzone.mjs auto --json  >> logs/auto.log
```

计划任务直接指向 `node.exe` 或 `.cmd` 会创建**可见的控制台窗口**，每 30 分钟闪一下。必须套一层 `wscript`，用 `Run(cmd, 0, False)`（`0` = `SW_HIDE`）。

> `.vbs` **必须纯 ASCII。** `wscript` 按 ANSI/GBK 读文件，UTF-8 中文注释的尾字节会吞掉行尾 CR，把下一行代码并进注释，表现为莫名其妙的 `缺少对象: 'shell'`。`writeLauncher()` 里用 `'ascii'` 编码写盘，就是为了这个。

8.2 其他平台

不自动改 crontab。别人的 crontab 是别人的东西，越界了。只返回一行建议让用户自己决定。

九、扩展

9.1 加一个新动作

1. 在 `actions.mjs` 里写一个函数，内部用 `withPage()` 包住（它负责开页、等加载、关浏览器）

2. 在 `run()` 的 switch 里加一个 case
3. 在 `qzone.mjs` 的命令分发里加一个 case（浏览器类命令走延迟 `import`，保证 `help` / `config` 秒回）
4. 补 `test/run.mjs`：纯逻辑部分一定要有测试
5. 更新 `SKILL.md` 和 `references/commands.md`

9.2 改选择器

两处：`READ_SCRIPT` 顶部的 `CARD_SELECTORS`，和 `likeByKeys()` 里的 `[class*="like_btn"]`。

别猜。先跑 `explore`，看 `counts` 和 `firstCard` 的真实 HTML。

9.3 不要加的东西

- **自动发说说 / 评论**：破坏性动作，一错就是社交事故
- **绕过风控**（换 IP、过验证码）：违反平台规则，而且会把账号搞废
- **读 cookie 导出**：等于凭空造一个凭证泄露面

十、测试

```
node test/run.mjs
```

32 项，不需要登录、不需要网络。覆盖：时间换算、规则判定、配置读写与类型转换、坏配置兜底、暂停开关。

测试用 `QZONE_HOME` 指向临时目录，不会碰真实配置。

> **没被覆盖的部分**：CDP 通信、DOM 抓取、点赞 —— 这些需要真实登录态，只能手动验证。改这两块之后至少跑一次 `feed` + `auto --dry-run`。

十一、已知限制

限制	原因	可能的解法
只能赞页面上已渲染的动态	不滚动就不在 DOM 里	`like` 加滚动参数；或先 `feed --scrolls N` 再连着赞
每轮都要启一次浏览器（约 3 秒）	无状态的 CLI 设计	常驻模式（`watch`）可以复用实例
页面改版会失效	DOM 选择器不是契约	`explore` 定位 + 改选择器
登录态会过期	腾讯的会话策略	重新扫码
没有评论/发说说	设计上不做破坏性动作	—
headless 可能被识别	UA 能改，但 JS 指纹改不干净	目前实测可用；真被封了把 `browser.headless` 设 0

十二、排错顺序

遇到问题按这个顺序，每一步排除一类原因：

1. `status` —— 登录态和开关
2. `explore` —— 页面打开了吗、结构对不对（带截图）
3. `feed` —— 抓取逻辑对不对
4. `auto --dry-run` —— 规则判定对不对
5. `auto` —— 真实点赞

本工具与腾讯公司无关。QQ 空间未开放此用途的接口，使用非官方途径存在账号风险，请自行评估。